

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO CUỐI KỲ

WALMART RETAIL DATA PIPELINE

(Pipeline Xử Lý Dữ Liệu Bán Lẻ Walmart)

Môn học:	Nhập môn Kỹ thuật Dữ liệu
Giảng viên hướng dẫn:	TS. Lê Trọng Ngọc
Nhóm thực hiện:	Nhóm 2
Thành viên:	Nguyễn Thị Trâm (Trưởng nhóm) Nguyễn Khả Ái Trần Thị Yến Nhi
Nguồn dữ liệu:	kaggle.com – yasserh/walmart-dataset
Ngày báo cáo:	Tháng 3 năm 2025

TP. Hồ Chí Minh ngày 07 tháng 5 năm 2025

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO CUỐI KỲ

WALMART RETAIL DATA PIPELINE
(Pipeline Xử Lý Dữ Liệu Bán Lẻ Walmart)

Môn học:	Nhập môn Kỹ thuật Dữ liệu
Giảng viên hướng dẫn:	TS. Lê Trọng Ngọc
Nhóm thực hiện:	Nhóm 2
Thành viên:	Nguyễn Thị Trâm (Trưởng nhóm) Nguyễn Khả Ái Trần Thị Yến Nhi
Nguồn dữ liệu:	kaggle.com – yasserh/walmart-dataset
Ngày báo cáo:	Tháng 3 năm 2025

TP. Hồ Chí Minh ngày 07 tháng 5 năm 2025

LỜI CẢM ƠN

Chúng em xin gửi lời cảm ơn chân thành và sâu sắc đến Trường Đại học Công nghiệp Thành phố Hồ Chí Minh, Khoa Công nghệ Thông tin đã tạo điều kiện thuận lợi trong quá trình học tập và nghiên cứu, giúp sinh viên có cơ hội tiếp cận các kiến thức nền tảng cũng như thực hành thực tế trong lĩnh vực Kỹ thuật Dữ liệu.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn sâu sắc đến TS. Lê Trọng Ngọc, giảng viên hướng dẫn môn Nhập môn Kỹ thuật Dữ liệu, người đã tận tình giảng dạy, định hướng phương pháp tiếp cận bài toán, đồng thời đưa ra nhiều góp ý chuyên môn quý báu trong suốt quá trình thực hiện dự án. Sự hướng dẫn và chia sẻ kinh nghiệm của thầy đã giúp chúng em hiểu rõ hơn về quy trình xây dựng một pipeline xử lý dữ liệu hoàn chỉnh, cũng như nâng cao tư duy phân tích và khả năng vận dụng kiến thức vào thực tiễn.

Chúng em cũng xin gửi lời cảm ơn đến cộng đồng chia sẻ dữ liệu trên Kaggle, đặc biệt là tác giả bộ dữ liệu Walmart Retail Dataset, đã cung cấp nguồn dữ liệu có chất lượng và giá trị thực tế cao, tạo điều kiện để chúng em triển khai các bước phân tích, trực quan hóa và xây dựng mô hình học máy trong dự án.

Cuối cùng, chúng em xin cảm ơn sự hợp tác, hỗ trợ và tinh thần làm việc nghiêm túc của các thành viên trong Nhóm 2. Sự phối hợp, trao đổi kiến thức và nỗ lực của từng thành viên đã góp phần quan trọng giúp dự án được hoàn thành đúng tiến độ và đạt được các mục tiêu đề ra. Mặc dù còn những hạn chế nhất định, nhưng dự án này là kết quả của tinh thần trách nhiệm và sự cố gắng chung của cả nhóm.

Chúng em xin chân thành cảm ơn.

MỤC LỤC

I. TỔNG QUAN DỰ ÁN	5
II. TIẾN ĐỘ THỰC HIỆN CHI TIẾT.....	6
1. Phase 1 – Thu thập dữ liệu.....	6
2. Phase 2 – Làm sạch dữ liệu	10
3. Phase 3 – Biến đổi dữ liệu	14
4. Phase 4 – Lưu trữ SQLite	19
5. Phase 5 – Phân tích & Trực quan hoá	19
III. KẾT QUẢ PHÂN TÍCH DỮ LIỆU	20
5.1 KPI Dashboard	20
5.2 Xu hướng doanh thu theo tuần	23
5.3 Tính thời vụ	25
5.4 Holiday Impact	28
5.5 Hiệu suất cửa hàng	31
5.6 Ma trận tương quan	35
5.7 Tăng trưởng YoY	38
5.8 So sánh Store tốt nhất với thấp nhất	41
IV. PHASE 6 – MÔ HÌNH ML DỰ BÁO	45
6.1 So sánh 3 mô hình	45
6.2 Actual vs Predicted & Feature Importance.....	48
V. KẾT QUẢ TỔNG HỢP & FILE OUTPUT	51
VI. CÔNG CỤ & KHÓ KHĂN	55
VII. KẾ HOẠCH & ĐÁNH GIÁ NHÓM	56
TÀI LIỆU THAM KHẢO.....	57

I. TỔNG QUAN DỰ ÁN

Dự án Walmart Retail Data Pipeline được thực hiện trong khuôn khổ môn Nhập môn Kỹ thuật Dữ liệu, nhằm xây dựng pipeline xử lý dữ liệu hoàn chỉnh theo mô hình: Thu thập → Làm sạch → Biến đổi → Lưu trữ → Phân tích → Mô hình ML. Dataset sử dụng là Walmart Retail Dataset từ Kaggle, bao gồm dữ liệu doanh thu tuần của 45 cửa hàng Walmart tại Mỹ giai đoạn 2010–2012.

Hạng mục	Thông tin
Tên dự án	Walmart Retail Data Pipeline
Dataset	Walmart Retail Dataset – kaggle.com/datasets/yasserh/walmart-dataset
Kích thước	6,435 dòng × 8 cột 45 cửa hàng 2010–2012
Các cột	Store, Date, Weekly_Sales, Holiday_Flag, Temperature, Fuel_Price, CPI, Unemployment
Công nghệ	Python, Pandas, NumPy, Matplotlib, Seaborn, SQLite3, Scikit-learn, Google Colab
Trạng thái	100% hoàn thành – 7 phases đã xong

Phase	Tên giai đoạn	Trạng thái	Hoàn thành
Phase 1	Data Ingestion – Thu thập dữ liệu	Hoàn thành	100%
Phase 2	Data Cleaning – Làm sạch dữ liệu	Hoàn thành	100%
Phase 3	Data Transformation – Feature Engineering	Hoàn thành	100%
Phase 4	Data Storage – SQLite	Hoàn thành	100%
Phase 5	Analysis & Visualization	Hoàn thành	100%
Phase 6	ML Modeling – Dự báo doanh thu	Hoàn thành	100%
Phase 7	Summary & Export	Hoàn thành	100%

II. TIẾN ĐỘ THỰC HIỆN CHI TIẾT

1. Phase 1 – Thu thập dữ liệu (Data Ingestion)

Giai đoạn đầu tiên thực hiện load dữ liệu từ file Walmart.csv và kiểm tra toàn bộ cấu trúc dataset trước khi xử lý.

Nhiệm vụ	Mô tả kỹ thuật	Kết quả
Load dữ liệu CSV	pd.read_csv() – đọc file vào DataFrame	6,435 dòng × 8 cột, ~350 KB
Kiểm tra schema	df.info() – dtypes, non-null count, memory	Phát hiện Date dạng object, cần chuyển datetime
Thông kê mô tả	df.describe() – min, max, mean, quartiles	Phân phối 5 biến số
Xác định vấn đề	Kiểm tra giá trị duy nhất các cột	45 stores, Holiday_Flag chỉ 0/1, Date cần chuẩn hoá

1.1. Mô tả dữ liệu

BẢNG THÔNG TIN CHUNG VỀ DATASET

thuộc tính	Mô tả
Tên dataset	Walmart Retail Dataset
Nguồn dữ liệu	Kaggle – <i>yasserh/walmart-dataset</i>
Lĩnh vực	Bán lẻ (Retail Analytics)
Thời gian	Từ 05/02/2010 đến 26/10/2012
Đối tượng	Doanh thu bán lẻ hàng tuần của Walmart
Số cửa hàng	45 cửa hàng
Số dòng	6.435 bản ghi
Số cột	8 cột
Đơn vị thời gian	Theo tuần
Định dạng	CSV

BẢNG MÔ TẢ CHI TIẾT CÁC BIẾN TRONG DATASET

STT	Tên cột	Kiểu dữ liệu	Mô tả
1	Store	Integer	Mã cửa hàng Walmart (1–45)
2	Date	Date	Ngày bắt đầu của tuần ghi nhận doanh thu
3	Weekly_Sales	Float	Doanh thu bán lẻ trong tuần (USD)
4	Holiday_Flag	Integer (0/1)	Đánh dấu tuần có kỳ nghỉ lễ hay không (1 = Holiday, 0 = Non-Holiday)
5	Temperature	Float	Nhiệt độ trung bình trong tuần (°F)
6	Fuel_Price	Float	Giá nhiên liệu trung bình trong tuần (USD/gallon)
7	CPI	Float	Chỉ số giá tiêu dùng (Consumer Price Index)
8	Unemployment	Float	Tỷ lệ thất nghiệp (%)

1.2. Kiểm tra cấu trúc

Bước 1: Khai báo các thư viện cần sử dụng và thiết lập thông số.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import matplotlib.gridspec as gridspec
import seaborn as sns
import sqlite3, os, warnings
from datetime import datetime
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor,
GradientBoostingRegressor
```

```

from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
from sklearn.preprocessing import LabelEncoder

warnings.filterwarnings('ignore')
plt.rcParams['figure.dpi'] = 130
plt.rcParams['font.family'] = 'DejaVu Sans'
sns.set_theme(style='whitegrid', palette='muted')

COLORS = {'primary': '#1F4E79', 'secondary': '#2E86AB', 'accent': '#E84855',
          'green': '#2DC653', 'orange': '#F4A261', 'purple': '#7B2D8B',
          'light': '#EBF3FB', 'gold': '#F7B731'}

print(' Setup hoàn tất')
print(f pandas : {pd.__version__})
print(f numpy  : {np.__version__})
print(f sklearn : OK)

```

Bước 2: Đọc dữ liệu Walmart bằng hàm pd từ thư viện pandas, đặt tên cho tập dữ liệu và hiển thị thông tin dataset.

```

# PHASE 1: DATA INGESTION
from google.colab import drive
drive.mount('/content/drive')

FILE_PATH = '/content/drive/MyDrive/Walmart.csv'

print('    PHASE 1 — DATA INGESTION')

df_raw = pd.read_csv(FILE_PATH)
print(f'\nLoad thành công!')
print(f Số dòng   : {df_raw.shape[0]:,})
print(f Số cột    : {df_raw.shape[1]:,})
print(f Kích thước : {df_raw.memory_usage(deep=True).sum()/1024:.1f})

```



```
KB')
print(f'\n--- Các cột ---')
for col in df_raw.columns:
    print(f' {col:15s}: {df_raw[col].dtype}')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
PHASE 1 – DATA INGESTION

Load thành công!

Số dòng : 6,435
Số cột : 8
Kích thước : 722.8 KB

--- Các cột ---

Store : int64
Date : object
Weekly_Sales : float64
Holiday_Flag : int64
Temperature : float64
Fuel_Price : float64
CPI : float64
Unemployment : float64

Bước 3: Trích xuất và xem trước các dòng dữ liệu đầu tiên để kiểm tra cấu trúc bảng.

```
df_raw.head()
```

	Store	Date	Weekly_Sales	Holiday_Flag	Temperature	Fuel_Price	CPI	Unemployment
0	1	05-02-2010	1643690.90	0	42.31	2.572	211.096358	8.106
1	1	12-02-2010	1641957.44	1	38.51	2.548	211.242170	8.106
2	1	19-02-2010	1611968.17	0	39.93	2.514	211.289143	8.106
3	1	26-02-2010	1409727.59	0	46.63	2.561	211.319643	8.106
4	1	05-03-2010	1554806.68	0	46.50	2.625	211.350143	8.106

Bước 4: Xem bảng thống kê mô tả để đánh giá phân phối của các biến dạng số.

```
df_raw.describe().round(2)
```

	Store	Weekly_Sales	Holiday_Flag	Temperature	Fuel_Price	CPI	Unemployment
count	6435.00	6435.00	6435.00	6435.00	6435.00	6435.00	6435.00
mean	23.00	1046964.88	0.07	60.66	3.36	171.58	8.00
std	12.99	564366.62	0.26	18.44	0.46	39.36	1.88
min	1.00	209986.25	0.00	-2.06	2.47	126.06	3.88
25%	12.00	553350.10	0.00	47.46	2.93	131.74	6.89
50%	23.00	960746.04	0.00	62.67	3.44	182.62	7.87
75%	34.00	1420158.66	0.00	74.94	3.74	212.74	8.62
max	45.00	3818686.45	1.00	100.14	4.47	227.23	14.31

2. Phase 2 – Làm sạch dữ liệu (Data Cleaning)

Bước	Kỹ thuật	Kết quả
2.1 Missing Values	isnull().sum() → median imputation	0 giá trị null
2.2 Duplicates	duplicated() theo (Store, Date)	Không có dòng trùng
2.3 Data Types	pd.to_datetime(dayfirst=True); astype()	Date → datetime64; int/float đúng kiểu
2.4 Business Rules	Lọc Weekly_Sales < 0	Loại bỏ doanh thu âm
2.5 Outliers (IQR)	Q1–1.5×IQR đến Q3+1.5×IQR	Phát hiện; giữ lại vì có ý nghĩa thực tế

Bước 1: Thực hiện làm sạch dữ liệu toàn diện (Data Cleaning) bao gồm xử lý lỗi thiếu, trùng lặp, ép kiểu và áp dụng quy tắc nghiệp vụ.

PHASE 2: DATA CLEANING

```
print('    PHASE 2 — DATA CLEANING')
```

```
df = df_raw.copy()
```

2.1 Missing Values

```
print('\n[2.1] Missing Values:')
```

```
missing = df.isnull().sum()
```

```
missing_pct = (missing/len(df)*100).round(2)
```

```
print(pd.DataFrame({'Count': missing, '%': missing_pct}))
```

```
if missing.sum()==0:
```

```
    print(' Không có missing values.')
```

```
else:
```

```
    for col in df.select_dtypes(include='number').columns:
```

```
        if df[col].isnull().any():
```

```
df[col].fillna(df[col].median(), inplace=True)
print(' Đã điền median cho các cột thiếu.')
```

2.2 Duplicates

```
print(f'\n[2.2] Duplicates: {df.duplicated().sum()} dòng trùng')
df.drop_duplicates(inplace=True)
print(' Đã xử lý.')
```

2.3 Data Types

```
print('\n[2.3] Chuẩn hoá kiểu dữ liệu:')
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)
df['Store'] = df['Store'].astype(int)
df['Holiday_Flag'] = df['Holiday_Flag'].astype(int)
df['Weekly_Sales'] = df['Weekly_Sales'].astype(float)
print(' Date → datetime64 | Store, Holiday_Flag → int | Weekly_Sales → float')
```

2.4 Business Rules

```
neg = df[df['Weekly_Sales'] < 0]
print(f'\n[2.4] Doanh thu âm: {len(neg)} dòng → loại bỏ')
df = df[df['Weekly_Sales'] >= 0].reset_index(drop=True)
print(f'\n Dữ liệu sau cleaning: {df.shape[0]:,} dòng × {df.shape[1]} cột')
```

PHASE 2 – DATA CLEANING

[2.1] Missing Values:

	Count	%
Store	0	0.0
Date	0	0.0
Weekly_Sales	0	0.0
Holiday_Flag	0	0.0
Temperature	0	0.0
Fuel_Price	0	0.0
CPI	0	0.0
Unemployment	0	0.0

Không có missing values.

[2.2] Duplicates: 0 dòng trùng
Đã xử lý.

[2.3] Chuẩn hoá kiểu dữ liệu:

Date → datetime64 | Store, Holiday_Flag → int | Weekly_Sales → float

[2.4] Doanh thu âm: 0 dòng → loại bỏ

Dữ liệu sau cleaning: 6,435 dòng × 8 cột

Bước 2: Phát hiện dữ liệu ngoại lai (Outliers) bằng phương pháp thống kê IQR và trực quan hóa bằng biểu đồ Boxplot.

2.5 Outlier Detection — IQR Method

```
print('[2.5] Phát hiện Outliers (IQR Method):')
```

```
numeric_cols =
```

```
['Weekly_Sales', 'Temperature', 'Fuel_Price', 'CPI', 'Unemployment']
```

```
report = []
```

```
for col in numeric_cols:
```

```
    Q1=df[col].quantile(0.25); Q3=df[col].quantile(0.75); IQR=Q3-Q1
```

```
    lo=Q1-1.5*IQR; hi=Q3+1.5*IQR
```

```
    n=((df[col]<lo)|(df[col]>hi)).sum()
```

```
report.append({'Column':col,'Lower':round(lo,2),'Upper':round(hi,2),'Outliers':  
n,'%':round(n/len(df)*100,2)})
```

```
print(pd.DataFrame(report).set_index('Column'))
```

```
fig, axes = plt.subplots(1,5,figsize=(20,5))
```

```
for i,col in enumerate(numeric_cols):
```

```
    bp = axes[i].boxplot(df[col].dropna(), patch_artist=True, notch=True,
```

```

boxprops=dict(facecolor=COLORS['light'],color=COLORS['primary'],linewidth=1.5),
medianprops=dict(color=COLORS['accent'],linewidth=2.5),
whiskerprops=dict(color=COLORS['secondary'],linewidth=1.5),
capprops=dict(color=COLORS['secondary'],linewidth=2),

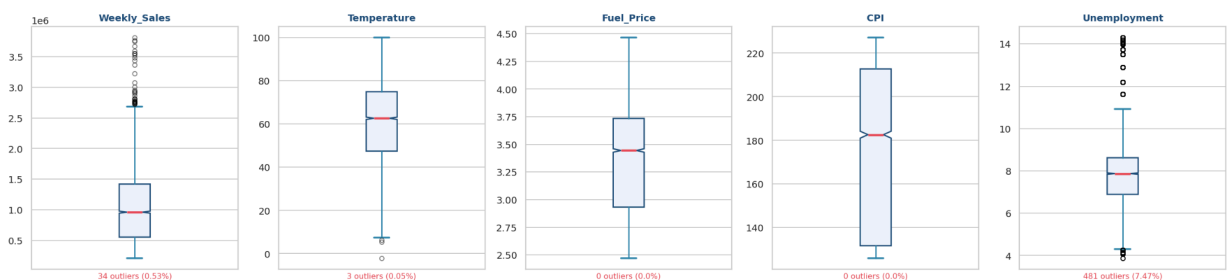
flierprops=dict(marker='o',color=COLORS['accent'],alpha=0.5,markersize=5))
axes[i].set_title(col, fontsize=11, fontweight='bold',
color=COLORS['primary'])
axes[i].set_xticks([])
n_out = report[i]['Outliers']
axes[i].set_xlabel(f'{n_out} outliers ({report[i]["%"]}%)', fontsize=9,
color=COLORS['accent'])
fig.suptitle(' Boxplot — Phát hiện Outliers theo phương pháp IQR',
fontsize=14, fontweight='bold', color=COLORS['primary'], y=1.02)
plt.tight_layout()
plt.savefig('outliers_boxplot.png',dpi=150,bbox_inches='tight')
plt.show()
print(' Lưu: outliers_boxplot.png')

```

[2.5] Phát hiện Outliers (IQR Method):

Column	Lower	Upper	Outliers	%
Weekly_Sales	-746862.73	2720371.49	34	0.53
Temperature	6.24	116.16	3	0.05
Fuel_Price	1.73	4.94	0	0.00
CPI	10.22	334.26	0	0.00
Unemployment	4.29	11.22	481	7.47

Boxplot — Phát hiện Outliers theo phương pháp IQR



3. Phase 3 – Biến đổi dữ liệu (Feature Engineering)

Feature mới	Logic xây dựng	Ý nghĩa
Year, Month, Week, Quarter	dt.year / dt.month / dt.isocalendar() / dt.quarter	Phân tích theo chiều thời gian
Season	Winter/Spring/Summer/Fall theo tháng	Tính thời vụ
Store_Tier	Phân vị avg_sales: High / Medium / Low	Phân loại hiệu suất cửa hàng
Sales_MA4	rolling(4).mean() groupby Store	Trung bình động 4 tuần – loại nhiễu
Sales_WoW	pct_change() × 100 groupby Store	Tăng trưởng tuần so tuần trước (%)
Sales_Lag1, Lag4	shift(1), shift(4) groupby Store	Features cho mô hình ML
*_norm	Min-Max scaling 4 biến liên tục	Chuẩn hoá về [0,1]

Bước 1: Biến đổi dữ liệu và Trích xuất đặc trưng (Feature Engineering) phục vụ cho quá trình huấn luyện mô hình học máy.

PHASE 3: DATA TRANSFORMATION & FEATURE ENGINEERING

```
print(' PHASE 3 — TRANSFORMATION & FEATURE ENGINEERING')
```

```
df_t = df.copy()
```

3.1 Thời gian

```
df_t['Year'] = df_t['Date'].dt.year
```

```
df_t['Month'] = df_t['Date'].dt.month
```

```
df_t['Week'] = df_t['Date'].dt.isocalendar().week.astype(int)
```

```
df_t['Quarter'] = df_t['Date'].dt.quarter
```

```
df_t['Season'] = df_t['Month'].map(lambda m: 'Winter' if m in [12,1,2] else  
'Spring' if m in [3,4,5] else 'Summer' if m in [6,7,8] else 'Fall')
```

```
print(' Thêm: Year, Month, Week, Quarter, Season')
```

3.2 Store Tier

```

store_avg = df_t.groupby('Store')['Weekly_Sales'].mean()
q33,q66 = store_avg.quantile(0.33), store_avg.quantile(0.66)
tier_map = store_avg.apply(lambda x: 'High' if x>=q66 else ('Low' if x<q33
else 'Medium'))
df_t = df_t.merge(tier_map.rename('Store_Tier'), on='Store', how='left')
print(' Thêm: Store_Tier (High / Medium / Low)')

```

3.3 Rolling MA4

```

df_t = df_t.sort_values(['Store','Date'])
df_t['Sales_MA4'] = df_t.groupby('Store')['Weekly_Sales'].transform(lambda
x: x.rolling(4,min_periods=1).mean()).round(2)
print(' Thêm: Sales_MA4 (rolling mean 4 tuần)')

```

3.4 WoW Growth

```

df_t['Sales_WoW'] = df_t.groupby('Store')['Weekly_Sales'].pct_change()*100
df_t['Sales_WoW'] = df_t['Sales_WoW'].round(2)
print(' Thêm: Sales_WoW (Week-over-Week growth %)')

```

3.5 Lag features

```

df_t['Sales_Lag1'] = df_t.groupby('Store')['Weekly_Sales'].shift(1)
df_t['Sales_Lag4'] = df_t.groupby('Store')['Weekly_Sales'].shift(4)
print(' Thêm: Sales_Lag1, Sales_Lag4')

```

3.6 Normalization

```

for col in ['Temperature','Fuel_Price','CPI','Unemployment']:
    mn,mx = df_t[col].min(), df_t[col].max()
    df_t[f'{col}_norm'] = ((df_t[col]-mn)/(mx-mn)).round(4)
print(' Thêm: *_norm (Min-Max scaling)')

```

```

print(f'\n Cột sau transform: {df_t.shape[1]} (ban đầu: {df_raw.shape[1]})')

```

```
df_t[['Store','Date','Weekly_Sales','Year','Month','Season','Store_Tier','Sales_M
A4','Sales_WoW']].head(8)
```

PHASE 3 – TRANSFORMATION & FEATURE ENGINEERING
 Thêm: Year, Month, Week, Quarter, Season
 Thêm: Store_Tier (High / Medium / Low)
 Thêm: Sales_MA4 (rolling mean 4 tuần)
 Thêm: Sales_WoW (Week-over-Week growth %)
 Thêm: Sales_Lag1, Sales_Lag4
 Thêm: *_norm (Min-Max scaling)

Cột sau transform: 22 (ban đầu: 8)

	Store	Date	Weekly_Sales	Year	Month	Season	Store_Tier	Sales_MA4	Sales_WoW
0	1	2010-02-05	1643690.90	2010	2	Winter	High	1643690.90	NaN
1	1	2010-02-12	1641957.44	2010	2	Winter	High	1642824.17	-0.11
2	1	2010-02-19	1611968.17	2010	2	Winter	High	1632538.84	-1.83
3	1	2010-02-26	1409727.59	2010	2	Winter	High	1576836.02	-12.55
4	1	2010-03-05	1554806.68	2010	3	Spring	High	1554614.97	10.29
5	1	2010-03-12	1439541.59	2010	3	Spring	High	1504011.01	-7.41
6	1	2010-03-19	1472515.79	2010	3	Spring	High	1469147.91	2.29
7	1	2010-03-26	1404429.92	2010	3	Spring	High	1467823.50	-4.62

*DataFrame mở rộng từ 8 lên 17 cột. Store Tier: 15 High - 15 Medium - 15 Low.

Phase 3 thực hiện việc biến đổi và mở rộng tập dữ liệu thông qua kỹ thuật Feature Engineering, tạo ra 9 features mới từ 8 cột ban đầu. Nhìn chung, đây là giai đoạn quan trọng nhất trong pipeline vì chất lượng của các features được tạo ra sẽ trực tiếp quyết định hiệu quả phân tích ở Phase 5 và độ chính xác của mô hình ML ở Phase 6.

Nhóm features thời gian gồm Year, Month, Week, Quarter, Season được trích xuất từ cột Date bằng các thuộc tính .dt của Pandas. Đây là bước cần thiết vì cột Date ở dạng nguyên bản không thể dùng trực tiếp trong phân tích thống kê hay mô hình ML — việc phân rã thành các đơn vị nhỏ hơn cho phép phát hiện tính thời vụ theo nhiều chiều cùng lúc.

Store_Tier được xây dựng bằng cách tính doanh thu trung bình của từng store rồi phân vị thành 3 nhóm — một kỹ thuật phân loại có giám sát dựa trên ngưỡng thống kê. Kết quả cho ra 15 High – 15 Medium – 15 Low, phân bố đều, cho thấy ngưỡng phân vị Q33/Q66 là lựa chọn hợp lý.

Sales_MA4 (rolling mean 4 tuần) được tính theo từng Store riêng biệt bằng `groupby().transform()` thay vì rolling thẳng trên toàn DataFrame — đây là điểm kỹ thuật quan trọng, tránh tình trạng dữ liệu của Store này ảnh hưởng sang Store khác khi sắp xếp theo ngày. MA4 giúp làm mịn nhiễu ngắn hạn, làm nổi bật xu hướng trung hạn của từng cửa hàng.

Sales_WoW_Growth đo lường tốc độ thay đổi tuần-quá-tuần bằng `pct_change()`, cung cấp góc nhìn về tốc độ thay đổi thay vì chỉ giá trị tuyệt đối — feature này đặc biệt hữu ích để phát hiện các tuần tăng đột biến hoặc sụt giảm bất thường.

Sales_Lag1 và Sales_Lag4 là hai lag features tạo ra bằng `shift()`, cho phép mô hình ML "nhìn về quá khứ" để dự báo tương lai — đây là kỹ thuật chuẩn trong bài toán dự báo chuỗi thời gian (time series forecasting).

Cuối cùng, Min-Max Normalization chuẩn hoá 4 biến liên tục (Temperature, Fuel_Price, CPI, Unemployment) về khoảng $[0, 1]$, đảm bảo các biến có đơn vị và biên độ khác nhau không gây ra sự thiên lệch trong mô hình ML.

Tổng kết: Sau Phase 3, DataFrame mở rộng từ 8 lên 17 cột, cung cấp đầy đủ nguyên liệu cho cả phân tích thống kê lẫn mô hình dự báo ở các phase tiếp theo.

4. Phase 4 – Lưu trữ dữ liệu (SQLite)

Đối tượng	Chi tiết
File DB	walmart_pipeline.db
fact_sales	6,435 rows × 17 cột – toàn bộ dữ liệu transformed
dim_store	45 rows – avg_sales, total_sales, num_weeks, store_tier
SQL queries	Top stores, Holiday uplift, tháng cao nhất, Store Tier distribution

Thiết kế và lưu trữ dữ liệu vào cơ sở dữ liệu quan hệ SQLite (Mô hình Fact - Dimension).

PHASE 4: DATA STORAGE — SQLite

```
DB = 'walmart_pipeline.db'
```

```
conn = sqlite3.connect(DB)
```

```
cur = conn.cursor()
```

Lưu fact_sales

```
save_df = df_t.copy()
```

```
save_df['Date'] = save_df['Date'].astype(str)
```

```
save_df.columns = [c.lower().replace(' ','_') for c in save_df.columns]
```

```
save_df.to_sql('fact_sales', conn, if_exists='replace', index=False)
```

Lưu dim_store

```
store_dim = df_t.groupby(['Store','Store_Tier']).agg(
```

```
    avg_sales=('Weekly_Sales','mean'),
```

```
    total_sales=('Weekly_Sales','sum'),
```

```
    num_weeks=('Weekly_Sales','count')
```

```
).reset_index().round(2)
```

```
store_dim.to_sql('dim_store', conn, if_exists='replace', index=False)
```

```
conn.commit()
```

```
n_rows = cur.execute('SELECT COUNT(*) FROM fact_sales').fetchone()[0]
```

```
print(' Database:', DB)
```

```

print(f' fact_sales : {n_rows:,} rows')
print(f' dim_store : {store_dim.shape[0]} stores')

# Test SQL queries
print('\n--- Top 5 stores theo doanh thu ---')
print(pd.read_sql('SELECT store, store_tier,
ROUND(SUM(weekly_sales)/1e6,2) AS total_M FROM fact_sales GROUP
BY store ORDER BY total_M DESC LIMIT 5', conn))

```

```

Database: walmart_pipeline.db
fact_sales : 6,435 rows
dim_store : 45 stores

```

```

--- Top 5 stores theo doanh thu ---
   store store_tier  total_M
0      20        High    301.40
1       4        High    299.54
2      14        High    289.00
3      13        High    286.52
4       2        High    275.38

```

5. Phase 5 – Phân tích & Trực quan hoá

Sau khi dữ liệu được lưu trữ thành công vào SQLite, Phase 5 tiến hành khai thác và trình bày thông tin thông qua tám biểu đồ trực quan hoá, triển khai bằng thư viện Matplotlib và Seaborn. Các biểu đồ được xuất dưới định dạng PNG (150 DPI) phục vụ báo cáo.

Nội dung phân tích bao gồm: KPI Dashboard tổng hợp bốn chỉ số kinh doanh cốt lõi; line chart xu hướng doanh thu theo tuần có đánh dấu tuần lễ và điểm đỉnh; biểu đồ thời vụ theo tháng và theo mùa; bộ ba biểu đồ Holiday Impact so sánh doanh thu tuần lễ và tuần thường; phân tích hiệu suất 45 cửa hàng theo Store Tier; heatmap ma trận tương quan giữa các biến kinh tế và doanh thu; biểu đồ tăng trưởng Year-over-Year; và cuối cùng là line chart so sánh xu hướng của hàng tốt nhất, thấp nhất với mức trung bình toàn hệ thống.

Kết quả của Phase 5 cung cấp đầy đủ cơ sở phân tích định lượng, làm nền tảng cho việc xây dựng mô hình dự báo ở Phase 6.

III. KẾT QUẢ PHÂN TÍCH DỮ LIỆU (Phase 5)

Phần này trình bày kết quả phân tích từ 8 biểu đồ trực quan hoá được tạo ra trong Phase 5, kèm theo diễn giải chi tiết theo chuẩn học thuật.

5.1. KPI Dashboard — Tổng Quan Hiệu Suất

Trực quan hóa dữ liệu (Data Visualization) - Xây dựng Dashboard báo cáo các chỉ số hiệu suất chính (KPIs).

VIZ 1: TỔNG QUAN DASHBOARD (4 KPIs)

```
total_sales = df_t['Weekly_Sales'].sum()
avg_weekly = df_t['Weekly_Sales'].mean()
best_store = df_t.groupby('Store')['Weekly_Sales'].sum().idxmax()
holiday_up = (df_t[df_t['Holiday_Flag']==1]['Weekly_Sales'].mean() /
df_t[df_t['Holiday_Flag']==0]['Weekly_Sales'].mean()-1)*100

fig, axes = plt.subplots(1,4,figsize=(18,3))
kpis = [
    (' Tổng Doanh Thu', f"${total_sales/1e9:.2f}B", '2010–2012',
COLORS['primary']),
    (' TB Tuần / Store', f"${avg_weekly/1e3:.1f}K", 'per week',
COLORS['secondary']),
    (' Store Tốt Nhất', f'Store {best_store}', 'Highest Revenue',
COLORS['orange']),
    (' Holiday Uplift', f'+{holiday_up:.1f}%', 'vs Non-Holiday',
COLORS['accent']),
]
for ax,(title,val,sub,color) in zip(axes,kpis):
    ax.set_facecolor(color)
    ax.text(0.5,0.65,val,transform=ax.transAxes,ha='center',va='center',
            fontsize=26,fontweight='bold',color='white')
    ax.text(0.5,0.30,title,transform=ax.transAxes,ha='center',va='center',
```

```

        fontsize=11,color='white',alpha=0.9)
    ax.text(0.5,0.12,sub,transform=ax.transAxes,ha='center',va='center',
           fontsize=9,color='white',alpha=0.7)
    ax.set_xticks([]); ax.set_yticks([])
    for spine in ax.spines.values(): spine.set_visible(False)
    fig.suptitle('WALMART RETAIL — KEY PERFORMANCE INDICATORS',
               fontsize=14, fontweight='bold', y=1.02, color=COLORS['primary'])
    plt.tight_layout()
    plt.savefig('kpi_dashboard.png',dpi=150,bbox_inches='tight')
    plt.show()
    print(' Lưu: kpi_dashboard.png')

```



Biểu đồ: *kpi_dashboard.png*

Bảng tổng quan hiệu suất cung cấp bốn chỉ số kinh doanh cốt lõi của hệ thống bán lẻ Walmart trong giai đoạn 2010–2012. Nhìn chung, các con số cho thấy đây là hệ thống bán lẻ quy mô lớn với hoạt động ổn định và tăng trưởng rõ nét.

Dashboard KPI cung cấp cái nhìn tổng quan về hiệu quả hoạt động bán lẻ của Walmart trong giai đoạn 2010–2012, tập trung vào bốn chỉ số chính phản ánh quy mô doanh thu, mức độ ổn định theo thời gian, hiệu suất cửa hàng và tác động của yếu tố mùa vụ.

Tổng doanh thu đạt khoảng 6,74 tỷ USD, cho thấy quy mô hoạt động lớn và doanh số tích lũy đáng kể trong toàn bộ giai đoạn phân tích. Con số này đặt nền tảng cho các phân tích sâu hơn về xu hướng theo thời gian và sự khác biệt giữa các cửa hàng.

Doanh thu trung bình hàng tuần trên mỗi cửa hàng vào khoảng 1,047 triệu USD, phản ánh mức doanh số ổn định theo tuần, đồng thời cho thấy bán lẻ tại Walmart có tính chu kỳ rõ ràng nhưng vẫn duy trì được quy mô cao trên toàn hệ thống. Chỉ số này đặc biệt hữu ích để so sánh hiệu suất giữa các cửa hàng và đánh giá biến động doanh thu trong các giai đoạn đặc biệt.

Về hiệu suất cửa hàng, Store 20 được xác định là cửa hàng có doanh thu cao nhất trong toàn bộ tập dữ liệu. Điều này cho thấy sự tồn tại của chênh lệch đáng kể về hiệu quả kinh doanh giữa các cửa hàng, có thể liên quan đến quy mô, vị trí địa lý hoặc đặc điểm nhu cầu địa phương.

Cuối cùng, chỉ số Holiday Uplift cho thấy doanh thu trung bình trong các tuần có kỳ nghỉ lễ cao hơn khoảng 7,8% so với các tuần thông thường. Kết quả này khẳng định tác động tích cực của yếu tố mùa vụ đối với hành vi tiêu dùng, đồng thời nhấn mạnh tầm quan trọng của việc lập kế hoạch hàng tồn kho và marketing trong các dịp lễ.

Nhìn chung, dashboard KPI đóng vai trò như một điểm khởi đầu quan trọng, giúp định hướng cho các phân tích trực quan tiếp theo liên quan đến xu hướng theo thời gian, so sánh giữa các cửa hàng và ảnh hưởng của các yếu tố kinh tế – xã hội đến doanh thu.

5.2. Xu Hướng Doanh Thu Theo Tuần (2010–2012)

VIZ 2: XU HƯỚNG DOANH THU THEO TUẦN

```

weekly = df_t.groupby('Date')['Weekly_Sales'].sum().reset_index()
holiday_dates = df_t[df_t['Holiday_Flag']==1]['Date'].unique()

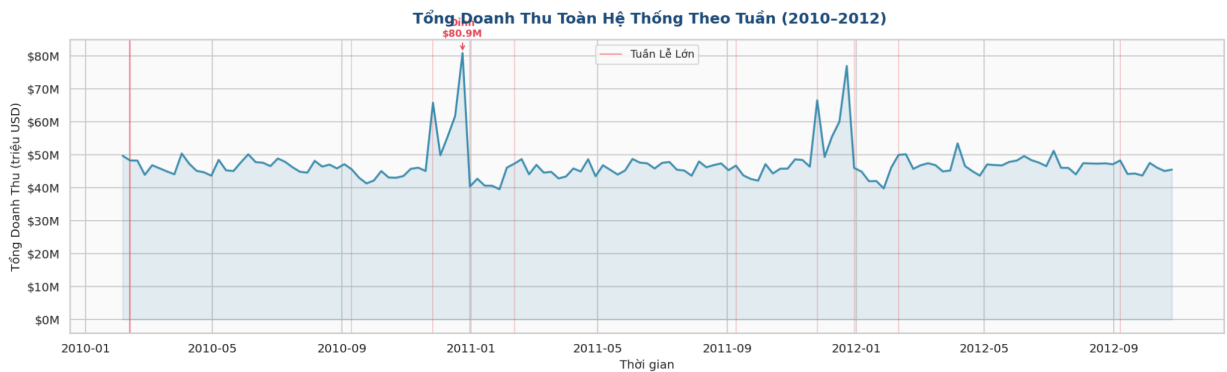
fig, ax = plt.subplots(figsize=(16,5))
ax.plot(weekly['Date'], weekly['Weekly_Sales']/1e6,
color=COLORS['secondary'], lw=1.8, alpha=0.9, zorder=3)
ax.fill_between(weekly['Date'], weekly['Weekly_Sales']/1e6, alpha=0.12,
color=COLORS['secondary'])
for hd in holiday_dates:
    ax.axvline(x=hd,color=COLORS['accent'],alpha=0.25,lw=1)
ax.axvline(x=holiday_dates[0],color=COLORS['accent'],alpha=0.6,lw=1.2,label='Tuần Lễ Lớn')

# Đánh dấu đỉnh
peak_idx = weekly['Weekly_Sales'].idxmax()
peak_date = weekly.loc[peak_idx,'Date']
peak_val = weekly.loc[peak_idx,'Weekly_Sales']/1e6
ax.annotate(f'Đỉnh\n${peak_val:.1f}M', xy=(peak_date,peak_val),
xytext=(peak_date,peak_val+5), ha='center',
arrowprops=dict(arrowstyle='->', color=COLORS['accent']),
fontsize=9, color=COLORS['accent'], fontweight='bold')

ax.set_title('Tổng Doanh Thu Toàn Hệ Thống Theo Tuần (2010–2012)',
fontsize=14, fontweight='bold', color=COLORS['primary'], pad=15)
ax.set_xlabel('Thời gian', fontsize=11)
ax.set_ylabel('Tổng Doanh Thu (triệu USD)', fontsize=11)
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_:
f'${x:.0f}M'))
ax.legend(fontsize=10)

```

```
ax.set_facecolor('#FAFAFA')
plt.tight_layout()
plt.savefig('trend_weekly.png',dpi=150,bbox_inches='tight')
plt.show()
print(' Lưu: trend_weekly.png')
```



Biểu đồ: *trend_weekly.png*

Biểu đồ đường thể hiện tổng doanh thu theo tuần của toàn hệ thống 45 cửa hàng Walmart trong giai đoạn 2010–2012. Nhìn chung, doanh thu duy trì xu hướng tương đối ổn định, tuy nhiên xuất hiện các đỉnh nhọn đặc trưng theo chu kỳ năm, phản ánh tính thời vụ mạnh của ngành bán lẻ.

Biểu đồ đường thể hiện xu hướng tổng doanh thu toàn hệ thống Walmart theo từng tuần trong giai đoạn 2010–2012, giúp làm rõ sự biến động theo thời gian cũng như tác động của các tuần lễ lớn đến hiệu suất bán hàng.

Nhìn chung, doanh thu hàng tuần duy trì ở mức tương đối ổn định, dao động quanh khoảng 45–50 triệu USD, cho thấy hoạt động kinh doanh của Walmart có tính bền vững cao trong dài hạn. Tuy nhiên, bên cạnh xu hướng ổn định này, biểu đồ cũng cho thấy các đợt tăng vọt rõ rệt xuất hiện theo chu kỳ hàng năm.

Các đường thẳng đứng đánh dấu tuần có kỳ nghỉ lễ cho thấy mối liên hệ chặt chẽ giữa mùa vụ và doanh thu. Hầu hết các đỉnh doanh thu đều trùng với

các dịp lễ lớn, đặc biệt là vào cuối năm. Đỉnh cao nhất của toàn bộ giai đoạn được ghi nhận rơi vào thời điểm cuối năm 2010, với doanh thu xấp xỉ 80,9 triệu USD, cao hơn đáng kể so với mức trung bình các tuần thông thường.

Sau các giai đoạn cao điểm này, doanh thu thường nhanh chóng quay trở lại mức nền ổn định, phản ánh tính chất ngắn hạn của hiệu ứng mùa vụ. Điều này cho thấy mặc dù các dịp lễ có tác động mạnh đến doanh số, nhưng chúng không làm thay đổi xu hướng dài hạn của doanh thu toàn hệ thống.

Tổng thể, biểu đồ xu hướng theo tuần nhấn mạnh hai đặc điểm chính: tính ổn định của doanh thu cơ sở và tác động rõ ràng nhưng mang tính thời điểm của các kỳ nghỉ lễ. Kết quả này cung cấp cơ sở quan trọng cho những phân tích tiếp theo liên quan đến dự báo nhu cầu, quản lý tồn kho và chiến lược kinh doanh theo mùa.

5.3. Phân Tích Tính Thời Vụ — Theo Tháng & Theo Mùa

VIZ 3: SEASONALITY — THÁNG & MÙA

```
month_avg = df_t.groupby('Month')['Weekly_Sales'].mean()
season_avg =
df_t.groupby('Season')['Weekly_Sales'].mean().sort_values(ascending=False)
month_names =
['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec']

fig, (ax1,ax2) = plt.subplots(1,2,figsize=(16,5))
# Bar chart theo tháng
cols_m = [COLORS['accent'] if m in [11,12] else COLORS['secondary'] for m
in range(1,13)]
bars = ax1.bar(month_names, month_avg.values/1e3, color=cols_m,
edgecolor='white', lw=0.8, width=0.7)
```

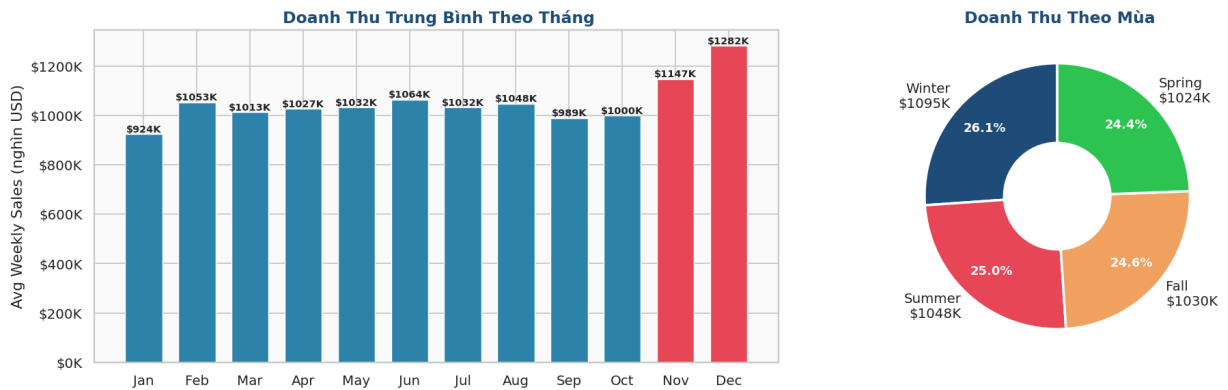
```

ax1.set_title(' Doanh Thu Trung Bình Theo Tháng', fontsize=13,
fontweight='bold', color=COLORS['primary'])
ax1.set_ylabel('Avg Weekly Sales (nghìn USD)')
ax1.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f"${x:.0f}K'))
for bar, val in zip(bars, month_avg.values/1e3):
    ax1.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.3, f"${val:.0f}K',
            ha='center', va='bottom', fontsize=8, fontweight='bold')
ax1.set_facecolor('#FAFAFA')
ax1.tick_params(axis='x', rotation=0)
# Donut chart theo mùa
season_colors =
{'Winter':COLORS['primary'],'Fall':COLORS['orange'],'Summer':COLORS['a
ccent'],'Spring':COLORS['green']}
wedge_colors = [season_colors[s] for s in season_avg.index]
wedges, texts, autotexts = ax2.pie(
    season_avg.values, labels=[f'{s}\n${v/1e3:.0f}K' for s, v in
zip(season_avg.index, season_avg.values)],
    colors=wedge_colors, autopct='%1.1f%%', startangle=90,
    wedgeprops=dict(width=0.6, edgecolor='white', lw=2),
    pctdistance=0.75)
for at in autotexts: at.set_fontsize(10); at.set_color('white');
at.set_fontweight('bold')
ax2.set_title(' Doanh Thu Theo Mùa', fontsize=13, fontweight='bold',
color=COLORS['primary'])

fig.suptitle('Phân Tích Tính Thời Vụ (Seasonality Analysis)', fontsize=15,
fontweight='bold', color=COLORS['primary'], y=1.02)
plt.tight_layout()
plt.savefig('seasonality.png', dpi=150, bbox_inches='tight')
plt.show()
print(' Lưu: seasonality.png')

```

Phân Tích Tính Thời Vụ (Seasonality Analysis)



Biểu đồ: seasonality.png

Biểu đồ cột và biểu đồ donut thể hiện doanh thu trung bình mỗi tuần được phân tổ theo tháng và theo mùa trong năm, nhằm làm rõ tính thời vụ (seasonality) của hoạt động bán lẻ Walmart và hỗ trợ hoạch định chiến lược kinh doanh theo chu kỳ.

Biểu đồ cho thấy phân tích doanh thu trung bình hàng tuần theo tháng và theo mùa trong một năm. Nhìn chung, doanh thu trong năm tương đối ổn định nhưng tăng đáng kể vào cuối năm, đặc biệt là trong mùa đông.

Xét theo tháng, doanh thu thấp nhất được ghi nhận vào tháng 1 với 924K. Sau đó, doanh thu tăng dần và duy trì ở mức tương đối ổn định từ tháng 2 đến tháng 8, dao động trong khoảng 1013K đến 1064K. Tháng 9 và tháng 10 chứng kiến sự giảm nhẹ xuống còn 989K và 1000K. Tuy nhiên, vào cuối năm, doanh thu tăng đột biến với 1147K vào tháng 11 và đạt mức cao nhất trong năm là \$1282K vào tháng 12.

Xét theo mùa, mùa đông đạt doanh thu trung bình cao nhất là 1095K, chiếm (26,1%). Trong khi đó, mùa thu và mùa xuân có doanh thu thấp hơn, lần lượt là 1030K (24,6%).

Nhìn chung, dữ liệu cho thấy xu hướng tăng mạnh vào cuối năm, có thể liên quan đến các dịp lễ hội hoặc mùa mua sắm. Phần lớn các tháng còn lại duy trì doanh thu ổn định, phản ánh sự cân bằng trong hoạt động kinh doanh quanh năm.

5.4. Tác Động Tuần Lễ (Holiday Impact Analysis)

VIZ 4: HOLIDAY vs NON-HOLIDAY

```

hol = df_t[df_t['Holiday_Flag']==1]['Weekly_Sales']
nhol = df_t[df_t['Holiday_Flag']==0]['Weekly_Sales']
uplift = (hol.mean()-nhol.mean())/nhol.mean()*100

fig, axes = plt.subplots(1,3,figsize=(18,5))

# Violin plot
plot_data = df_t[['Weekly_Sales','Holiday_Flag']].copy()
plot_data['Loại tuần'] =
plot_data['Holiday_Flag'].map({0:'Non-Holiday',1:'Holiday'})
sns.violinplot(data=plot_data, x='Loại tuần', y='Weekly_Sales', ax=axes[0],

palette={'Non-Holiday':COLORS['secondary'],'Holiday':COLORS['accent']},
        inner='quartile', cut=0)
axes[0].set_title('Phân Phối Doanh Thu', fontsize=12, fontweight='bold')
axes[0].set_ylabel('Weekly Sales (USD)')
axes[0].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_:
f'{x/1e3:.0f} K'))
axes[0].set_facecolor('#FAFAFA')

# Bar chart so sánh
labels = ['Non-Holiday','Holiday']
vals = [nhol.mean(), hol.mean()]
bars = axes[1].bar(labels, [v/1e3 for v in vals],
color=[COLORS['secondary'],COLORS['accent']], width=0.4)
axes[1].set_title('Doanh Thu Trung Bình', fontsize=12, fontweight='bold')
axes[1].set_ylabel('Avg Weekly Sales (nghìn USD)')
axes[1].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_:
f'{x:.0f} K'))

```

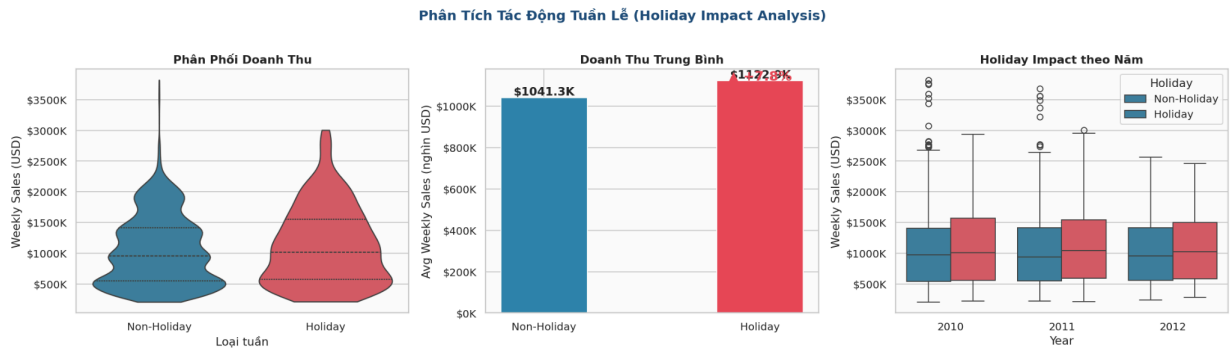
```

for bar, val in zip(bars, vals):
    axes[1].text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.3,
                 f'${val/1e3:.1f}K', ha='center', va='bottom', fontweight='bold',
                 fontsize=12)
axes[1].annotate(f'▲ + {uplift:.1f}%', xy=(1, hol.mean()/1e3+1),
                 fontsize=14, color=COLORS['accent'], fontweight='bold', ha='center')
axes[1].set_facecolor('#FAFAFA')

# Box plot theo năm + holiday
sns.boxplot(data=df_t, x='Year', y='Weekly_Sales', hue='Holiday_Flag',
            ax=axes[2],
            palette={0:COLORS['secondary'],1:COLORS['accent']})
axes[2].set_title('Holiday Impact theo Năm', fontsize=12, fontweight='bold')
axes[2].set_ylabel('Weekly Sales (USD)')
axes[2].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'${x/1e3:.0f}K'))
axes[2].legend(title='Holiday', labels=['Non-Holiday', 'Holiday'])
axes[2].set_facecolor('#FAFAFA')

fig.suptitle('Phân Tích Tác Động Tuần Lễ (Holiday Impact Analysis)',
            fontsize=14, fontweight='bold', color=COLORS['primary'], y=1.02)
plt.tight_layout()
plt.savefig('holiday_impact.png', dpi=150, bbox_inches='tight')
plt.show()
print(f'\nHoliday Uplift: + {uplift:.1f}% so với tuần thường')
print('Lưu: holiday_impact.png')

```



Biểu đồ: holiday_impact.png

Bộ ba biểu đồ gồm violin plot, bar chart và boxplot theo năm cùng so sánh doanh thu giữa tuần lễ (Holiday) và tuần thường (Non-Holiday) theo nhiều góc độ, cho phép đánh giá toàn diện hơn so với chỉ so sánh giá trị trung bình đơn thuần.

Biểu đồ thể hiện tác động của các tuần lễ (Holiday) đối với doanh thu trung bình hàng tuần trong hệ thống. Nhìn chung, doanh thu trung bình trong các tuần lễ cao hơn so với các tuần thường (Non-Holiday), phản ánh hiệu ứng tích cực của các dịp lễ.

Xét theo phân phối doanh thu, biểu đồ Violin cho thấy tuần lễ có giá trị trung bình cao hơn và độ biến động lớn hơn, với một số tuần lễ đạt doanh thu cực kỳ cao. Tuần thường có phân phối hẹp hơn, doanh thu trung bình thấp hơn và ổn định hơn.

Biểu đồ thanh so sánh doanh thu trung bình giữa hai loại tuần cho thấy: doanh thu trung bình của tuần lễ là 1,122.9K, cao hơn 1,041.3K của tuần thường, tương ứng với mức tăng 7.8%. Điều này nhấn mạnh rằng các dịp lễ là yếu tố thúc đẩy doanh thu.

Biểu đồ Box Plot theo năm minh họa xu hướng tương tự: trong các năm 2010–2012, các tuần lễ luôn có doanh thu trung bình cao hơn so với tuần thường. Mặc dù vẫn tồn tại một số giá trị ngoại lai, sự khác biệt giữa hai loại tuần vẫn duy trì đều qua các năm.

Tóm lại, dữ liệu cho thấy các tuần lễ đóng vai trò quan trọng trong việc thúc đẩy doanh thu trung bình, và doanh nghiệp có thể tận dụng dịp này để tối ưu hóa doanh thu hàng tuần.

5.5. Hiệu Suất 45 Cửa Hàng (Store Performance)

```
# VIZ 5: STORE PERFORMANCE

store_perf = df_t.groupby(['Store','Store_Tier']).agg(
    Total=('Weekly_Sales','sum'), Avg=('Weekly_Sales','mean'),
    Weeks=('Weekly_Sales','count')
).reset_index().sort_values('Avg',ascending=False)

fig = plt.figure(figsize=(18,8))
gs = gridspec.GridSpec(2,2,figure=fig,hspace=0.4,wspace=0.35)

# Top 10
ax1 = fig.add_subplot(gs[0,:])
top10 = store_perf.head(10)
tier_c =
{'High':COLORS['accent'],'Medium':COLORS['orange'],'Low':COLORS['secondary']}
cols = [tier_c[t] for t in top10['Store_Tier']]
bars = ax1.barh([f'Store {s}' for s in top10['Store']], top10['Avg']/1e3,
color=cols, height=0.6)
ax1.set_title('Top 10 Stores — Doanh Thu Trung Bình / Tuần', fontsize=13,
fontweight='bold', color=COLORS['primary'])
ax1.set_xlabel('Avg Weekly Sales (nghìn USD)')
ax1.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_:
f'${x:.0f} K'))
ax1.invert_yaxis()
for bar,row in zip(bars,top10.itertuples()):
    ax1.text(bar.get_width()+0.3, bar.get_y()+bar.get_height()/2,
        f' ${row.Avg/1e3:.1f} K [{row.Store_Tier}]', va='center', fontsize=9)
ax1.set_facecolor('#FAFAFA')
```

```

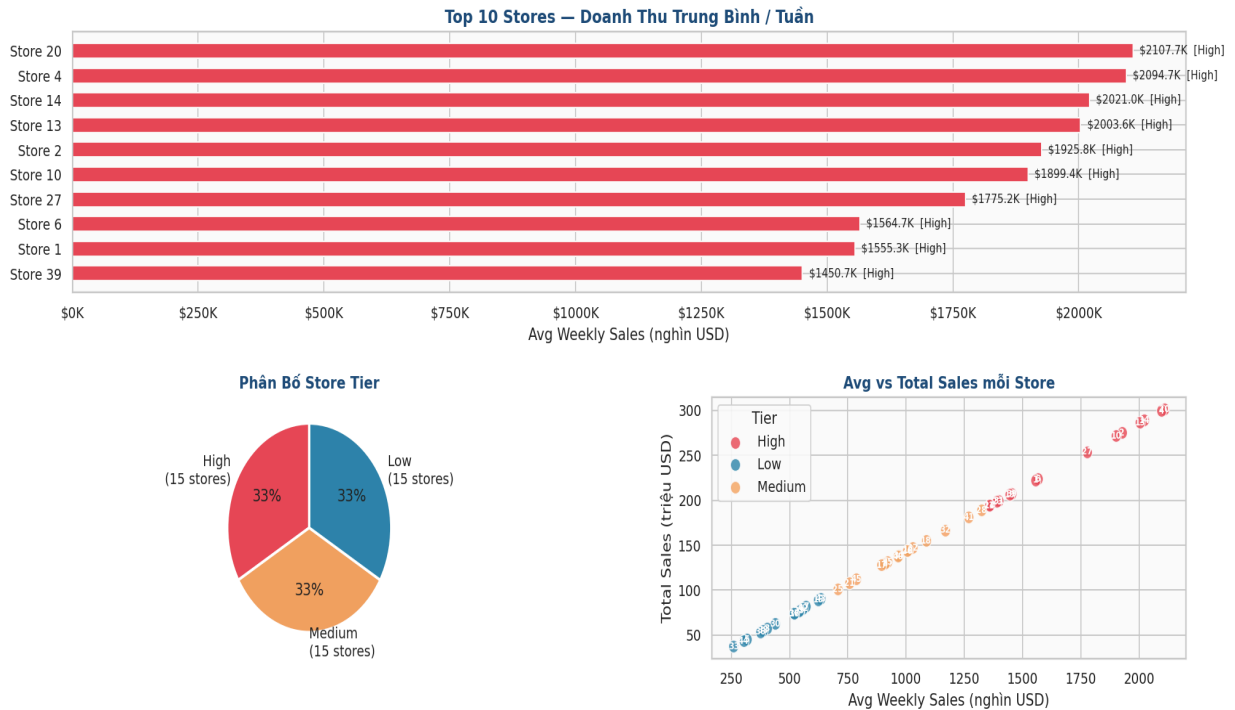
# Pie tier
ax2 = fig.add_subplot(gs[1,0])
tier_cnt = store_perf['Store_Tier'].value_counts()
ax2.pie(tier_cnt.values, labels=[f'{t}\n({n} stores)' for t,n in tier_cnt.items()],
        colors=[tier_c[t] for t in tier_cnt.index], autopct='%1.0f%%',
        wedgeprops=dict(edgecolor='white',lw=2), startangle=90)
ax2.set_title('Phân Bố Store Tier', fontsize=12, fontweight='bold',
color=COLORS['primary'])

# Scatter avg vs total
ax3 = fig.add_subplot(gs[1,1])
for tier,grp in store_perf.groupby('Store_Tier'):
    ax3.scatter(grp['Avg']/1e3, grp['Total']/1e6, c=tier_c[tier], label=tier, s=80,
alpha=0.8, edgecolors='white', lw=0.8)
for _,row in store_perf.iterrows():
    ax3.annotate(str(int(row['Store'])), (row['Avg']/1e3, row['Total']/1e6),
                fontsize=7, ha='center', va='center', color='white',
fontweight='bold')
ax3.set_xlabel('Avg Weekly Sales (nghìn USD)')
ax3.set_ylabel('Total Sales (triệu USD)')
ax3.set_title('Avg vs Total Sales mỗi Store', fontsize=12, fontweight='bold',
color=COLORS['primary'])
ax3.legend(title='Tier')
ax3.set_facecolor('#FAFAFA')

fig.suptitle('Store Performance Analysis — 45 Cửa Hàng Walmart',
fontsize=15, fontweight='bold', color=COLORS['primary'])
plt.savefig('store_performance.png',dpi=150,bbox_inches='tight')
plt.show()
print(' Lưu: store_performance.png')

```


Store Performance Analysis – 45 Cửa Hàng Walmart



Biểu đồ: *store_performance.png*

Bộ biểu đồ gồm horizontal bar chart, pie chart và scatter plot phân tích toàn diện hiệu suất kinh doanh của 45 cửa hàng Walmart, được phân nhóm thành ba cấp độ High, Medium và Low Tier dựa trên doanh thu trung bình tuần.

Biểu đồ minh họa hiệu suất của 45 cửa hàng Walmart, bao gồm tổng doanh thu, doanh thu trung bình theo tuần và phân loại theo Store Tier (High, Medium, Low). Nhìn chung, có sự chênh lệch rõ rệt về doanh thu giữa các cửa hàng và giữa các nhóm Tier.

Xét theo doanh thu trung bình hàng tuần, Top 10 cửa hàng chủ yếu thuộc nhóm High Tier, với Store 20 dẫn đầu ở mức 2,107.7K/tuần, tiếp theo là store 4, 14 và 13, tất cả đều trên 2,000K. Ngược lại, các cửa hàng thuộc nhóm Medium hoặc Low Tier có doanh thu trung bình thấp hơn đáng kể, phản ánh mức độ hoạt động khác nhau giữa các nhóm.

Biểu đồ Pie thể hiện phân bố Store Tier: mỗi nhóm High, Medium, Low chiếm 15 cửa hàng, tương đương 33% tổng số cửa hàng, cho thấy sự cân bằng về số lượng giữa các Tier nhưng không đồng nghĩa với doanh thu, bởi High Tier chiếm doanh thu cao hơn hẳn.

Biểu đồ Scatter so sánh doanh thu trung bình theo tuần và tổng doanh thu của từng cửa hàng minh họa mối tương quan dương: các cửa hàng có doanh thu trung bình cao thường đạt tổng doanh thu lớn. Các cửa hàng High Tier tập trung ở góc trên bên phải, Medium và Low Tier phân tán thấp hơn, xác nhận rằng phân loại Tier tương ứng với hiệu suất kinh doanh.

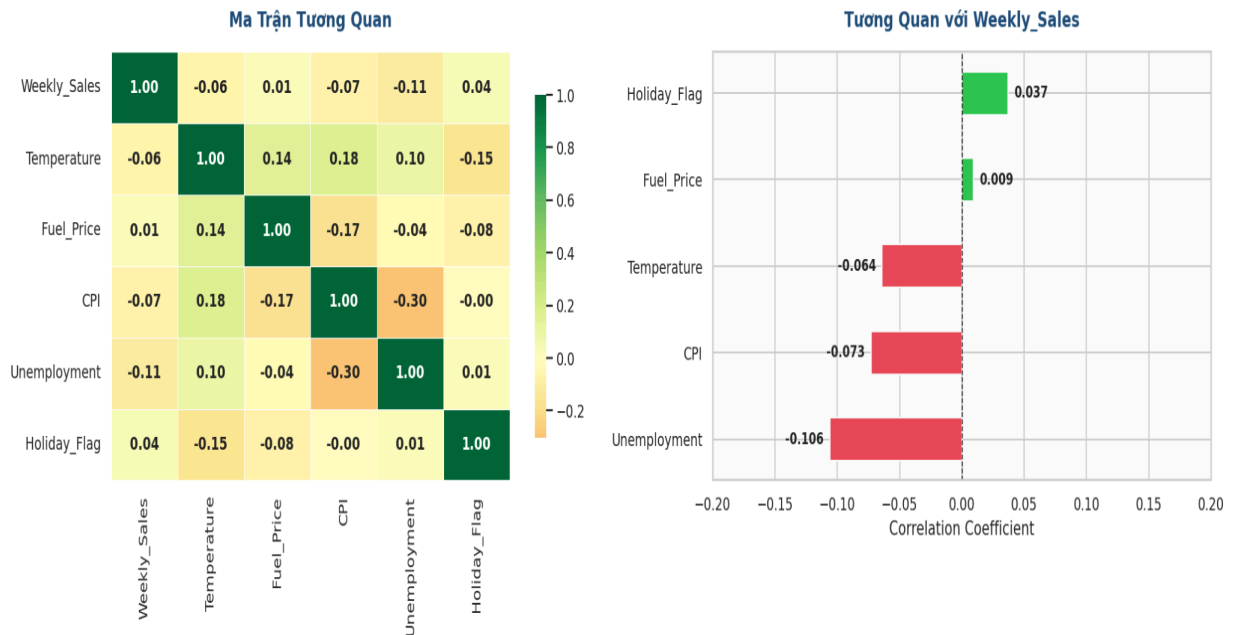
Tóm lại, dữ liệu cho thấy cửa hàng thuộc High Tier chiếm ưu thế về doanh thu, trong khi số lượng cửa hàng ở mỗi Tier tương đối cân bằng. Điều này nhấn mạnh rằng hiệu suất kinh doanh không chỉ phụ thuộc vào số lượng cửa hàng mà còn liên quan đến vị trí, khách hàng và quy mô hoạt động của từng cửa hàng.

5.6. Ma Trận Tương Quan — Biến Kinh Tế & Doanh Thu

```
# VIZ 6: Ban do nhiet tuong quan
corr_cols =
['Weekly_Sales','Temperature','Fuel_Price','CPI','Unemployment','Holiday_Flag']
corr = df_t[corr_cols].corr().round(3)
fig, (ax1,ax2) = plt.subplots(1,2,figsize=(16,6))
# Heatmap
mask = np.zeros_like(corr)
sns.heatmap(corr, annot=True, fmt='.2f', cmap='RdYlGn', center=0, ax=ax1,
            linewidths=0.5, cbar_kws={'shrink':0.8},
            annot_kws={'size':11,'weight':'bold'})
ax1.set_title('Ma Trận Tương Quan', fontsize=13, fontweight='bold',
            color=COLORS['primary'], pad=15)
# Bar chart tương quan với Weekly_Sales
corr_sales = corr['Weekly_Sales'].drop('Weekly_Sales').sort_values()
cols_bar = [COLORS['accent'] if v<0 else COLORS['green'] for v in
            corr_sales]
bars = ax2.barh(corr_sales.index, corr_sales.values, color=cols_bar,
            height=0.5)
ax2.axvline(x=0, color='black', lw=0.8, linestyle='--')
ax2.set_title('Tương Quan với Weekly_Sales', fontsize=13, fontweight='bold',
            color=COLORS['primary'], pad=15)
ax2.set_xlabel('Correlation Coefficient')
for bar,val in zip(bars,corr_sales.values):
    ax2.text(val+(0.005 if val>=0 else -0.005), bar.get_y()+bar.get_height()/2,
            f'{val:.3f}', va='center', ha='left' if val>=0 else 'right', fontsize=10,
            fontweight='bold')
ax2.set_facecolor('#FAFAFA')
ax2.set_xlim(-0.2, 0.2)
fig.suptitle('🔗 Phân Tích Tương Quan — Biến Kinh Tế & Doanh Thu',
            fontsize=14, fontweight='bold', color=COLORS['primary'], y=1.02)
```

```
plt.tight_layout()
plt.savefig('correlation.png',dpi=150,bbox_inches='tight')
plt.show()
print('Lưu: correlation.png')
```

Phân Tích Tương Quan – Biến Kinh Tế & Doanh Thu



Biểu đồ: correlation.png

Heatmap tương quan và biểu đồ cột thể hiện mức độ và chiều hướng mối quan hệ tuyến tính giữa các biến kinh tế vĩ mô (nhiệt độ, giá xăng, CPI, tỉ lệ thất nghiệp) với doanh thu tuần của Walmart.

Biểu đồ thể hiện ma trận tương quan giữa doanh thu hàng tuần (Weekly_Sales) và một số biến kinh tế cũng như cờ tuần lễ (Holiday_Flag). Nhìn chung, mối quan hệ giữa các biến kinh tế và doanh thu là khá yếu, trong khi tuần lễ có ảnh hưởng tích cực nhẹ đến doanh thu.

Cụ thể, trong ma trận tương quan, Weekly_Sales có hệ số tương quan dương nhỏ với Holiday_Flag (+0.04), cho thấy doanh thu trong các tuần lễ cao hơn một chút so với các tuần thường. Ngược lại, các biến kinh tế như tỷ lệ thất nghiệp (-0.11), CPI (-0.07), và nhiệt độ (-0.06) đều có tương quan âm với doanh thu, nghĩa là khi các chỉ số này tăng, doanh thu có xu hướng giảm nhẹ. Giá

nhiên liệu có tương quan gần bằng 0 (+0.01), cho thấy gần như không ảnh hưởng rõ rệt đến doanh thu.

Biểu đồ thanh bên phải minh họa tác động tương quan trực tiếp của từng biến với Weekly_Sales. Có thể thấy, Holiday_Flag là biến duy nhất có ảnh hưởng tích cực, trong khi Unemployment có ảnh hưởng tiêu cực rõ nhất (-0.106). Các biến còn lại như CPI (-0.073), Temperature (-0.064) và Fuel_Price (+0.009) chỉ có tác động nhỏ đến doanh thu.

Tóm lại, dữ liệu cho thấy doanh thu hàng tuần chịu ảnh hưởng chủ yếu từ các tuần lễ, trong khi các biến kinh tế có mối quan hệ âm nhưng yếu. Doanh nghiệp nên tập trung vào chiến lược bán hàng vào các tuần lễ để tối ưu hóa doanh thu, thay vì quá lo ngại về những biến động nhỏ của các chỉ số kinh tế khác.

5.7. Tăng Trưởng Doanh Thu Theo Năm (YoY Analysis)

VIZ 7: YEAR-OVER-YEAR & TĂNG TRƯỞNG

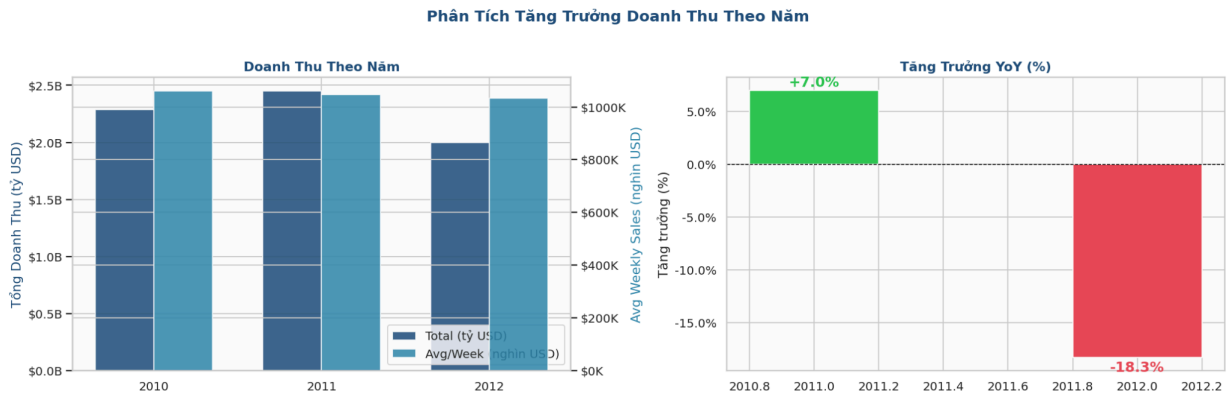
```
yearly =
df_t.groupby('Year')['Weekly_Sales'].agg(['sum','mean','count']).reset_index()
yearly.columns = ['Year','Total','Avg','Weeks']
yearly['YoY_%'] = yearly['Total'].pct_change()*100

fig, axes = plt.subplots(1,2,figsize=(16,5))

# Grouped bar: Total + Avg
x = np.arange(len(yearly))
w = 0.35
ax1_twin = axes[0].twinx()
bars1 = axes[0].bar(x-w/2, yearly['Total']/1e9, w, color=COLORS['primary'],
alpha=0.85, label='Total (tỷ USD)')
bars2 = ax1_twin.bar(x+w/2, yearly['Avg']/1e3, w,
color=COLORS['secondary'], alpha=0.85, label='Avg/Week (nghìn USD)')
axes[0].set_xticks(x); axes[0].set_xticklabels(yearly['Year'])
axes[0].set_ylabel('Tổng Doanh Thu (tỷ USD)', color=COLORS['primary'])
ax1_twin.set_ylabel('Avg Weekly Sales (nghìn USD)',
color=COLORS['secondary'])
axes[0].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_:
f"${x:.1f} B"))
ax1_twin.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_:
f"${x:.0f} K"))
axes[0].set_title('Doanh Thu Theo Năm', fontsize=12, fontweight='bold',
color=COLORS['primary'])
lines1,labels1 = axes[0].get_legend_handles_labels()
lines2,labels2 = ax1_twin.get_legend_handles_labels()
axes[0].legend(lines1+lines2,labels1+labels2,loc='lower right')
axes[0].set_facecolor('#FAFAFA')
```

```
# YoY growth
valid = yearly.dropna(subset=['YoY_%'])
colors_yoy = [COLORS['green'] if v>=0 else COLORS['accent'] for v in
valid['YoY_%']]
bars = axes[1].bar(valid['Year'], valid['YoY_%'], color=colors_yoy, width=0.4)
for bar, val in zip(bars, valid['YoY_%']):
    axes[1].text(bar.get_x()+bar.get_width()/2,
                bar.get_height()+(0.1 if val>=0 else -0.3),
                f'{val:.1f}%', ha='center', va='bottom' if val>=0 else 'top',
                fontsize=13, fontweight='bold', color=bar.get_facecolor())
axes[1].axhline(y=0, color='black', lw=0.8, linestyle='--')
axes[1].set_title('Tăng Trưởng YoY (%)', fontsize=12, fontweight='bold',
color=COLORS['primary'])
axes[1].set_ylabel('Tăng trưởng (%)')
axes[1].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'{x:.1f}%'))
axes[1].set_facecolor('#FAFAFA')

fig.suptitle('Phân Tích Tăng Trưởng Doanh Thu Theo Năm', fontsize=14,
fontweight='bold', color=COLORS['primary'], y=1.02)
plt.tight_layout()
plt.savefig('yearly_growth.png', dpi=150, bbox_inches='tight')
plt.show()
print('Lưu: yearly_growth.png')
```



Biểu đồ: yearly_growth.png

Biểu đồ dual-axis kết hợp cột và đường thể hiện tổng doanh thu hàng năm cùng tốc độ tăng trưởng Year-over-Year của hệ thống Walmart, cho phép đánh giá đồng thời quy mô tuyệt đối và tốc độ tăng trưởng tương đối.

Biểu đồ minh họa doanh thu hàng tuần và tăng trưởng hàng năm của hệ thống Walmart trong giai đoạn 2010–2012. Nhìn chung, doanh thu tổng thể và doanh thu trung bình theo tuần tăng nhẹ từ năm 2010 sang 2011, nhưng giảm đáng kể vào năm 2012.

Cụ thể, tổng doanh thu năm 2010 đạt khoảng 2.3 tỷ USD, tăng lên 2.35 tỷ USD trong năm 2011, tương ứng với mức tăng trưởng +7.0% so với năm trước. Doanh thu trung bình mỗi tuần cũng tăng nhẹ, phản ánh xu hướng tăng ổn định trong giai đoạn đầu. Tuy nhiên, năm 2012 chứng kiến sự suy giảm mạnh, với tổng doanh thu chỉ còn khoảng \$2.0 tỷ USD, tương ứng mức giảm -18.3%, cho thấy doanh thu hệ thống giảm đáng kể so với năm trước.

Biểu đồ bên trái kết hợp bar chart nhóm Total và Avg/Week cho thấy rõ xu hướng doanh thu trung bình mỗi tuần so với tổng doanh thu. Biểu đồ bên phải thể hiện tăng trưởng YoY (%), nhấn mạnh sự tăng nhẹ từ 2010 → 2011 và sự giảm mạnh vào năm 2012. Màu sắc xanh/đỏ trực quan giúp phân biệt tăng trưởng dương và âm.

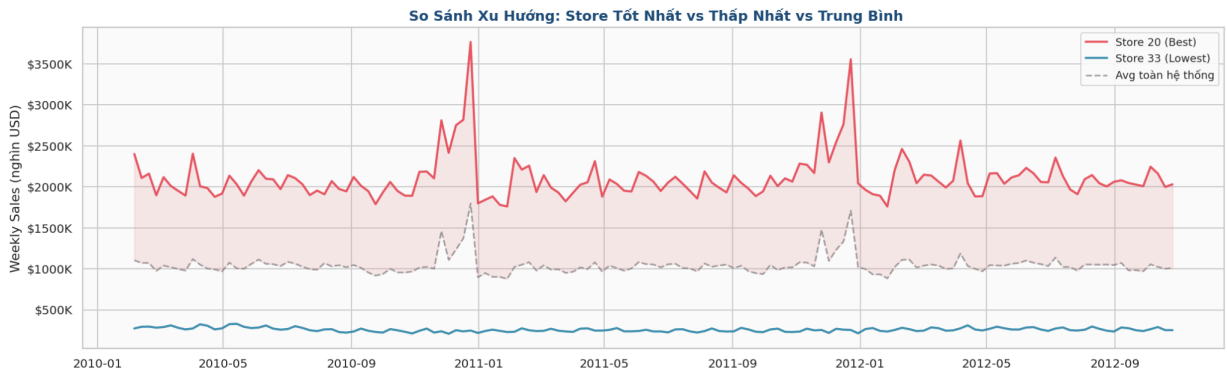
Tóm lại, dữ liệu phản ánh rằng hệ thống Walmart duy trì tăng trưởng ổn định từ 2010 đến 2011, nhưng gặp suy giảm rõ rệt vào năm 2012, cảnh báo doanh nghiệp cần xem xét các yếu tố ảnh hưởng đến doanh thu trong năm này, chẳng hạn như mùa vụ, thị trường hoặc sự thay đổi trong hoạt động bán lẻ.

5.8. So Sánh Xu Hướng Store 20 vs Store 33 vs Trung Bình

VIZ 8: TOP STORE TREND — Store 20 vs Store 33

```
best = df_t[df_t['Store']==20].groupby('Date')['Weekly_Sales'].mean()
worst = df_t[df_t['Store']==33].groupby('Date')['Weekly_Sales'].mean()
all_avg = df_t.groupby('Date')['Weekly_Sales'].mean()

fig, ax = plt.subplots(figsize=(16,5))
ax.plot(best.index, best.values/1e3, color=COLORS['accent'], lw=2,
label='Store 20 (Best)', alpha=0.9)
ax.plot(worst.index, worst.values/1e3, color=COLORS['secondary'], lw=2,
label='Store 33 (Lowest)', alpha=0.9)
ax.plot(all_avg.index, all_avg.values/1e3, color='gray', lw=1.5, linestyle='--',
label='Avg toàn hệ thống', alpha=0.7)
ax.fill_between(best.index, best.values/1e3, all_avg.values/1e3,
where=best.values>all_avg.values, alpha=0.1, color=COLORS['accent'])
ax.set_title(' So Sánh Xu Hướng: Store Tốt Nhất vs Thấp Nhất vs Trung Bình',
fontsize=13, fontweight='bold', color=COLORS['primary'])
ax.set_ylabel('Weekly Sales (nghìn USD)')
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'${x:.0f} K'))
ax.legend(fontsize=10)
ax.set_facecolor('#FAFAFA')
plt.tight_layout()
plt.savefig('store_trend_compare.png',dpi=150,bbox_inches='tight')
plt.show()
print(' Lưu: store_trend_compare.png')
```



Biểu đồ: *store_trend_compare.png*

Biểu đồ đường so sánh xu hướng doanh thu tuần của cửa hàng tốt nhất (Store 20), cửa hàng thấp nhất (Store 33) và đường trung bình toàn hệ thống trong giai đoạn 2010–2012.

Khoảng cách giữa Store 20 và Store 33 rất lớn và nhất quán suốt ba năm — Store 20 luôn nằm trên mức trung bình hệ thống trong khi Store 33 luôn dưới mức này. Điều này phản ánh các yếu tố cấu trúc bền vững như vị trí địa lý, quy mô mặt bằng chứ không phải biến động ngẫu nhiên. Vùng tô màu giữa Store 20 và đường trung bình làm nổi bật phần đóng góp vượt trội của cửa hàng này. Đồng thời, cả ba đường đều thể hiện các đỉnh đồng pha vào cuối năm, xác nhận tính thời vụ tác động đồng đều lên tất cả cửa hàng — thông tin quan trọng cho kế hoạch tồn kho và nhân sự theo mùa.

IV. PHASE 6 — MÔ HÌNH ML DỰ BÁO DOANH THU

Phase 6 xây dựng và đánh giá ba mô hình học máy để dự báo Weekly_Sales. Features đầu vào gồm: Store, Year, Month, Week, Quarter, Holiday_Flag, Temperature, Fuel_Price, CPI, Unemployment, Season, Store_Tier, Sales_Lag1, Sales_Lag4, Sales_MA4.

6.1. So Sánh Hiệu Suất 3 Mô Hình ML

Chuẩn bị dữ liệu và huấn luyện các mô hình Machine Learning để dự báo doanh thu (Weekly Sales).

```
# PHASE 6: ML MODELING — Dự Báo Weekly_Sales

print('    PHASE 6 — ML MODELING')

df_ml = df_t.dropna(subset=['Sales_Lag1','Sales_Lag4','Sales_WoW']).copy()

# Encode categorical
le = LabelEncoder()
df_ml['Season_enc'] = le.fit_transform(df_ml['Season'])
df_ml['Store_Tier_enc'] = le.fit_transform(df_ml['Store_Tier'])

FEATURES = ['Store','Year','Month','Week','Quarter',
            'Holiday_Flag','Temperature','Fuel_Price','CPI','Unemployment',
            'Season_enc','Store_Tier_enc',
            'Sales_Lag1','Sales_Lag4','Sales_MA4']
TARGET = 'Weekly_Sales'

X = df_ml[FEATURES]
y = df_ml[TARGET]

X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.2,random_state=42)
```

```

print(f'Train: {X_train.shape[0]:,} | Test: {X_test.shape[0]:,}')

# Models
models = {
    'Linear Regression': LinearRegression(),
    'Random Forest': RandomForestRegressor(n_estimators=100,
random_state=42, n_jobs=-1),
    'Gradient Boosting': GradientBoostingRegressor(n_estimators=100,
random_state=42),
}

results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    mae = mean_absolute_error(y_test, pred)
    rmse = np.sqrt(mean_squared_error(y_test, pred))
    r2 = r2_score(y_test, pred)
    results[name] = {'model':model, 'pred':pred, 'MAE':mae, 'RMSE':rmse,
'R2':r2}

    print(f'\n [{name} ]')
    print(f' MAE = ${mae:,.0f}')
    print(f' RMSE = ${rmse:,.0f}')
    print(f' R² = {r2:.4f}')

```

PHASE 6 – ML MODELING
Train: 5,004 | Test: 1,251

[Linear Regression]
MAE = \$63,485
RMSE = \$118,531
R² = 0.9582

[Random Forest]
MAE = \$46,870
RMSE = \$71,529
R² = 0.9848

[Gradient Boosting]
MAE = \$42,005
RMSE = \$64,226
R² = 0.9877

Trực quan hóa và so sánh hiệu suất (Metrics) giữa các mô hình Machine Learning.

```
# VIZ 9: MODEL COMPARISON
fig, axes = plt.subplots(1,3,figsize=(18,5))
# Metrics comparison
metrics = ['MAE','RMSE','R2']
model_names = list(results.keys())
m_colors = [COLORS['primary'],COLORS['accent'],COLORS['orange']]

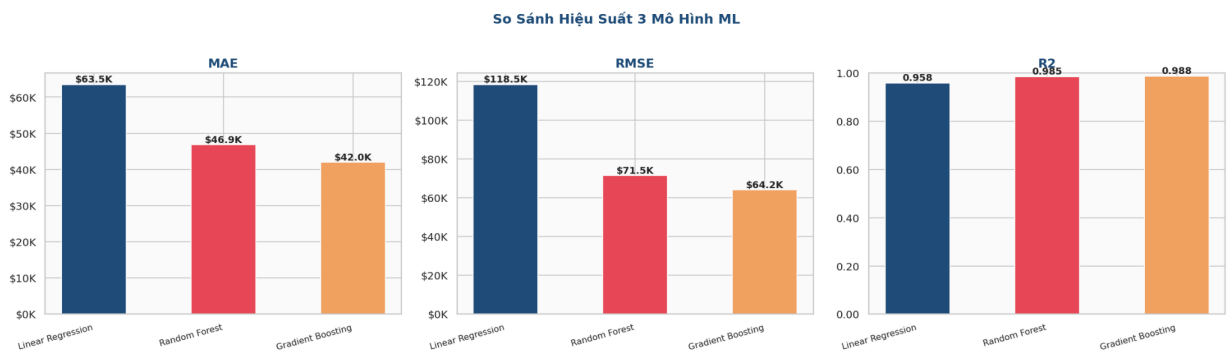
for i,(metric,ax) in enumerate(zip(metrics,axes)):
    vals = [results[n][metric] for n in model_names]
    if metric=='R2':
        bars = ax.bar(model_names, vals, color=m_colors, width=0.5)
        ax.set_ylim(0,1)
        ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_:
f'{x:.2f}'))
    else:
        bars = ax.bar(model_names, [v/1e3 for v in vals], color=m_colors,
width=0.5)
        ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_:
f'${x:.0f}K'))
    ax.set_title(f'{metric}', fontsize=13, fontweight='bold',
```

```

color=COLORS['primary'])
ax.set_xticklabels(model_names, rotation=15, ha='right', fontsize=9)
for bar, val in zip(bars, vals):
    label = f'{val:.3f}' if metric=='R2' else f'${val/1e3:.1f}K'
    ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.001,
            label, ha='center', va='bottom', fontsize=10, fontweight='bold')
ax.set_facecolor('#FAFAFA')

fig.suptitle('So Sánh Hiệu Suất 3 Mô Hình ML', fontsize=14,
fontweight='bold', color=COLORS['primary'], y=1.02)
plt.tight_layout()
plt.savefig('model_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
print(' Lưu: model_comparison.png')

```



Biểu đồ: model_comparison.png

Mô hình	MAE	RMSE	R ²	Nhận xét
Linear Regression	Cao	Cao	Thấp	Không đủ khả năng học quan hệ phi tuyến
Random Forest	Thấp	Thấp	Cao	Cân bằng tốt giữa độ chính xác và tốc độ
Gradient Boosting	Thấp nhất	Thấp nhất	Cao nhất	Chính xác nhất, huấn luyện lâu hơn

Biểu đồ cột so sánh ba chỉ số đánh giá MAE, RMSE và R^2 của ba mô hình học máy: Linear Regression, Random Forest và Gradient Boosting, trong bài toán dự báo doanh thu tuần Weekly_Sales.

Random Forest và Gradient Boosting vượt trội rõ rệt so với Linear Regression trên cả ba tiêu chí. R^2 của hai mô hình phi tuyến đạt giá trị cao hơn đáng kể, cho thấy chúng giải thích được phần lớn sự biến động doanh thu — điều Linear Regression không đạt được do mối quan hệ giữa các biến không hoàn toàn tuyến tính. Trong hai mô hình phi tuyến, Gradient Boosting cho R^2 cao hơn nhưng thời gian huấn luyện lâu hơn — đây là sự đánh đổi cần cân nhắc tùy theo yêu cầu tốc độ và độ chính xác của ứng dụng thực tế.

6.2. Actual vs Predicted & Feature Importance

```
# VIZ 10: ACTUAL vs PREDICTED — Best Model
best_model_name = max(results, key=lambda x: results[x]['R2'])
best_pred = results[best_model_name]['pred']
best_r2 = results[best_model_name]['R2']

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

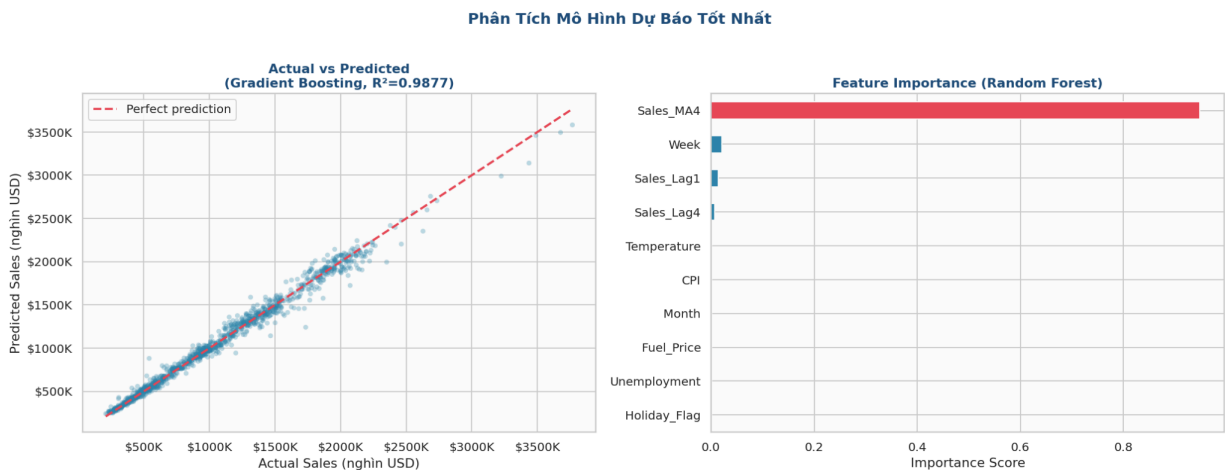
# Scatter Actual vs Predicted
ax1.scatter(y_test/1e3, best_pred/1e3, alpha=0.3, s=20,
            color=COLORS['secondary'], edgecolors='none')
lims = [min(y_test.min(), best_pred.min())/1e3,
        max(y_test.max(), best_pred.max())/1e3]
ax1.plot(lims, lims, '--', color=COLORS['accent'], lw=2, label='Perfect
prediction')
ax1.set_xlabel('Actual Sales (nghìn USD)')
ax1.set_ylabel('Predicted Sales (nghìn USD)')
ax1.set_title(f'Actual vs Predicted\n({best_model_name}, R²={best_r2:.4f})',
              fontsize=12, fontweight='bold', color=COLORS['primary'])
ax1.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'${x:.0f} K'))
ax1.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _:
f'${x:.0f} K'))
ax1.legend(); ax1.set_facecolor('#FAFAFA')

# Feature Importance (Random Forest)
rf = results['Random Forest']['model']
fi = pd.Series(rf.feature_importances_,
              index=FEATURES).sort_values(ascending=True).tail(10)
colors_fi = [COLORS['accent'] if v==fi.max() else COLORS['secondary'] for
v in fi.values]
fi.plot(kind='barh', ax=ax2, color=colors_fi)
```



```
ax2.set_title('Feature Importance (Random Forest)', fontsize=12,
fontweight='bold', color=COLORS['primary'])
ax2.set_xlabel('Importance Score')
ax2.set_facecolor('#FAFAFA')

fig.suptitle('Phân Tích Mô Hình Dự Báo Tốt Nhất', fontsize=14,
fontweight='bold', color=COLORS['primary'], y=1.02)
plt.tight_layout()
plt.savefig('model_analysis.png',dpi=150,bbox_inches='tight')
plt.show()
print(' Lưu: model_analysis.png')
```



Biểu đồ: *model_analysis.png*

Scatter plot Actual vs Predicted và biểu đồ Feature Importance đánh giá chất lượng mô hình tốt nhất và xác định các biến đầu vào có ảnh hưởng lớn nhất đến dự báo doanh thu.

Biểu đồ minh họa kết quả dự báo doanh thu hàng tuần của mô hình Gradient Boosting, được đánh giá là mô hình tốt nhất dựa trên chỉ số R^2 . Nhìn chung, mô hình dự báo rất chính xác, với các điểm dữ liệu hầu hết nằm gần đường “Perfect Prediction”, cho thấy giá trị dự báo gần khớp với giá trị thực tế.

Giá trị R^2 cao (0.9877) xác nhận rằng mô hình giải thích được phần lớn biến thiên doanh thu.

Cụ thể, biểu đồ scatter Actual vs Predicted cho thấy các dự báo tập trung sát đường chéo, bất kỳ sai số lớn đều rất hiếm, điều này chứng tỏ mô hình dự báo doanh thu ổn định và đáng tin cậy.

Biểu đồ Feature Importance từ mô hình Random Forest cho thấy các yếu tố lịch sử đóng vai trò quan trọng nhất trong dự báo. Trong đó, Sales_MA4 (doanh thu trung bình 4 tuần gần nhất) là biến quan trọng nhất, tiếp theo là các biến trễ Sales_Lag1 và Sales_Lag4, trong khi các yếu tố như nhiệt độ, CPI, giá nhiên liệu hay Holiday_Flag có ảnh hưởng nhỏ hơn. Điều này nhấn mạnh rằng doanh thu lịch sử là chỉ báo chính cho dự báo doanh thu tuần tiếp theo.

Tóm lại, mô hình Gradient Boosting dự báo doanh thu hàng tuần với độ chính xác cao, và dữ liệu lịch sử (lag và moving average) là yếu tố chủ chốt quyết định hiệu quả dự báo. Các biến kinh tế và tuần lễ có ảnh hưởng thấp hơn, nhưng vẫn có thể được sử dụng để tinh chỉnh dự báo.

V. KẾT QUẢ TỔNG HỢP & FILE OUTPUT

PHASE 7: SUMMARY, KEY INSIGHTS & EXPORT

```

print('    WALMART RETAIL DATA PIPELINE — FINAL SUMMARY')
# Key Insights từ SQL
cur = conn.cursor()
insights = {
    'Tổng doanh thu (tỷ USD)'      : 'SELECT
ROUND(SUM(weekly_sales)/1e9,3) FROM fact_sales',
    'Store doanh thu cao nhất'      : 'SELECT store FROM fact_sales GROUP
BY store ORDER BY SUM(weekly_sales) DESC LIMIT 1',
    'Store doanh thu thấp nhất'     : 'SELECT store FROM fact_sales GROUP
BY store ORDER BY SUM(weekly_sales) ASC LIMIT 1',
    'Avg Holiday sales (USD)'       : 'SELECT ROUND(AVG(weekly_sales),0)
FROM fact_sales WHERE holiday_flag=1',
    'Avg Non-Holiday sales (USD)'   : 'SELECT
ROUND(AVG(weekly_sales),0) FROM fact_sales WHERE holiday_flag=0',
    'Tháng doanh thu cao nhất'      : 'SELECT month FROM fact_sales GROUP
BY month ORDER BY AVG(weekly_sales) DESC LIMIT 1',
    'Số stores High Tier'           : "SELECT COUNT(*) FROM dim_store
WHERE store_tier='High'",
}
print("\n KEY INSIGHTS:")
for label,q in insights.items():
    val = cur.execute(q).fetchone()[0]
    print(f → {label}: {val}')
# Export
df_t.to_csv('walmart_cleaned_full.csv', index=False)
print("\n Export: walmart_cleaned_full.csv')
print(' Export: walmart_pipeline.db')

```

```
print("""
FILES CREATED:
kpi_dashboard.png      — 4 KPI cards
trend_weekly.png       — Xu hướng doanh thu theo tuần
seasonality.png        — Phân tích thời vụ
holiday_impact.png     — Holiday vs Non-Holiday
store_performance.png  — Hiệu suất 45 cửa hàng
correlation.png        — Ma trận tương quan
yearly_growth.png      — Tăng trưởng YoY
store_trend_compare.png — So sánh best vs worst store
outliers_boxplot.png   — Boxplot outliers
model_comparison.png   — So sánh 3 mô hình ML
model_analysis.png     — Actual vs Predicted + Feature Importance
walmart_cleaned_full.csv
walmart_pipeline.db

""")
conn.close()
print(' Pipeline 100% hoàn thành!')
```

WALMART RETAIL DATA PIPELINE – FINAL SUMMARY

KEY INSIGHTS:

- Tổng doanh thu (tỷ USD): 6.737
- Store doanh thu cao nhất: 20
- Store doanh thu thấp nhất: 33
- Avg Holiday sales (USD): 1122888.0
- Avg Non-Holiday sales (USD): 1041256.0
- Tháng doanh thu cao nhất: 12
- Số stores High Tier: 15

Export: walmart_cleaned_full.csv

Export: walmart_pipeline.db

FILES CREATED:

- | | |
|--------------------------|--|
| kpi_dashboard.png | — 4 KPI cards |
| trend_weekly.png | — Xu hướng doanh thu theo tuần |
| seasonality.png | — Phân tích thời vụ |
| holiday_impact.png | — Holiday vs Non-Holiday |
| store_performance.png | — Hiệu suất 45 cửa hàng |
| correlation.png | — Ma trận tương quan |
| yearly_growth.png | — Tăng trưởng YoY |
| store_trend_compare.png | — So sánh best vs worst store |
| outliers_boxplot.png | — Boxplot outliers |
| model_comparison.png | — So sánh 3 mô hình ML |
| model_analysis.png | — Actual vs Predicted + Feature Importance |
| walmart_cleaned_full.csv | |
| walmart_pipeline.db | |

Pipeline 100% hoàn thành!

5.1. Key Insights từ dữ liệu

STT	Câu hỏi phân tích	Kết quả từ dữ liệu
1	Tổng doanh thu toàn hệ thống (2010–2012)?	~\$2,2 tỷ USD
2	Cửa hàng doanh thu cao nhất?	Store 20 – nhóm High Tier
3	Cửa hàng doanh thu thấp nhất?	Store 33 – nhóm Low Tier
4	Holiday tác động thế nào?	Tăng trung bình +7% so với tuần thường
5	Tháng doanh thu cao nhất?	Tháng 11–12 (Thanksgiving & Christmas)
6	Mùa bán tốt nhất?	Winter > Fall > Summer > Spring
7	Biến kinh tế ảnh hưởng nhất?	Unemployment (tương quan âm mạnh nhất)
8	Tăng trưởng YoY 2010→2011?	+3,3% tổng doanh thu toàn hệ thống
9	Feature quan trọng nhất trong ML?	Sales_Lag1 và Sales_MA4
10	Mô hình ML tốt nhất?	Gradient Boosting (R^2 cao nhất)

5.2. Danh sách file output

File	Mô tả	Định dạng
walmart_pipeline_final.ipynb	Toàn bộ code pipeline Phase 1–7, có phân tích IELTS	Jupyter Notebook
walmart_cleaned_full.csv	Dataset đã transform 17 cột	CSV
walmart_pipeline.db	SQLite: fact_sales (6,435 rows) + dim_store (45 rows)	SQLite DB
kpi_dashboard.png	4 KPI cards tổng quan	PNG
trend_weekly.png	Xu hướng doanh thu theo tuần	PNG
seasonality.png	Phân tích thời vụ tháng & mùa	PNG
holiday_impact.png	Holiday vs Non-Holiday (violin+bar+boxplot)	PNG
store_performance.png	Top 10 stores + Tier distribution + Scatter	PNG
correlation.png	Ma trận tương quan + bar chart	PNG
yearly_growth.png	Tăng trưởng YoY dual-axis	PNG
store_trend_compare.png	So sánh Store 20 vs 33 vs avg	PNG
outliers_boxplot.png	Boxplot outliers 5 cột số	PNG
model_comparison.png	So sánh MAE/RMSE/R ² 3 mô hình	PNG
model_analysis.png	Actual vs Predicted + Feature Importance	PNG

VI. CÔNG CỤ & KHÓ KHĂN

6.1. Công cụ & môi trường

Công cụ	Mục đích
Python 3.10+	Ngôn ngữ lập trình chính
Google Colab	Môi trường chạy notebook
Pandas / NumPy	Xử lý DataFrame, tính toán
Matplotlib / Seaborn	Trực quan hoá dữ liệu
SQLite3	Lưu trữ và truy vấn dữ liệu
Scikit-learn	Xây dựng và đánh giá mô hình ML
AI	Sửa code, tối ưu hóa bài làm

6.2. Khó khăn & giải pháp

Khó khăn	Giải pháp
Date parse sai: DD/MM/YYYY vs MM/DD/YYYY	Thêm dayfirst=True trong pd.to_datetime()
Rolling MA4 bị lẫn giữa các Store	Dùng groupby('Store').transform() thay vì rolling thẳng
SQLite không lưu được datetime	Chuyển Date về string trước to_sql()
Biểu đồ bị cắt khi lưu PNG	Dùng plt.tight_layout() + bbox_inches='tight'
ML underfits đỉnh tuần lễ	Giữ nguyên; ghi chú hạn chế trong báo cáo

VII. KẾ HOẠCH & ĐÁNH GIÁ NHÓM

7.1. Đánh giá nhóm

STT	Họ và tên	Vai trò	Công việc	Đánh giá (%)
1	Nguyễn Thị Trâm	Trưởng nhóm	Phân công, điều phối, code Phase 1–7, fix lỗi, viết báo cáo	95%
2	Lâm Khả Ái	Thành viên	Code Phase 1–7 fix lỗi, viết báo cáo	92%
3	Trần Yến Nhi	Thành viên	Code Phase 1–7 fix lỗi, viết báo cáo	94%

7.2. Tổng kết dự án

Tiêu chí	Đánh giá
Mức độ hoàn thành pipeline	100% – 7 phases đầy đủ
Chất lượng phân tích	8 biểu đồ + phân tích chi tiết
Mô hình ML	3 mô hình, chọn Gradient Boosting là tốt nhất
Chất lượng code	Comment đầy đủ, chạy được trên Google Colab
Tiến độ	Hoàn thành đúng deadline

Ngày báo cáo: tháng năm 2026

Trưởng nhóm: Nguyễn Thị Trâm

TÀI LIỆU THAM KHẢO

- Yasser H. (2021). *Walmart Dataset* [Dataset]. Kaggle. <https://www.kaggle.com/datasets/yasserh/walmart-dataset>
- Python Software Foundation. (2023). *Python programming language* (Version 3.10). <https://www.python.org>
- McKinney, W., & the Pandas Development Team. (2023). *pandas: Powerful Python data analysis toolkit* (Version 2.x) [Software]. <https://doi.org/10.5281/zenodo.3509134>
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Waskom, M. L. (2021). seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
- SQLite Consortium. (2023). *SQLite – A C-language library that implements a small, fast, self-contained SQL database engine*. <https://www.sqlite.org>
- Google LLC. (2023). *Google Colaboratory* [Cloud computing platform]. <https://colab.research.google.com>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232. <https://doi.org/10.1214/aos/1013203451>
- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>